



15th Lesson

Speed Layer & Stream Processing

- **Realtime Interactive/Ad-hoc Analytics**, users issue ad-hoc dynamic queries and the system responds interactively.
- **Realtime Streaming Analytics**, users issue static queries once and they do not change, and the system processes data as they come in **without storing**. Stream Processing technologies enable streaming analytics.

Event-driven Applications



- Fraud detection
- Anomaly detection
- Rule-based alerting
- Business process monitoring
- Web application (social networking)
- ...

Realtime Streaming Analytics



- Processing data as they come in without storing them and react to those data very fast, often within few milliseconds.
- Such technologies are not new.
 - History goes back to Active Databases (2000+),
 - Stream processing (e.g. Aurora (2003) , Borealis (2005+) and later Apache Storm),
 - Distributed Streaming Operators(2005),
 - Complex Event processing.
- Counting usecases are only the tip of the iceberg of real-life realtime usecases.
- Since the input data arrives as a data stream, a ***time dimension*** is always present in the incoming data.

Why is stream processing need?



- **Reason 1:** Some data naturally comes as a never-ending stream of events. Stream processing **naturally fit** with time series data and detecting patterns over time.

Why is stream processing need?



- **Reason 1:** Some data naturally comes as a never-ending stream of events. Stream processing **naturally fit** with time series data and detecting patterns over time.
- **Reason 2:** Batch processing lets the data build up and try to process them at once while stream processing process data as they come in hence spread the processing over time. Hence stream processing can work with **a lot less hardware than batch processing**.

Why is stream processing need?



- **Reason 1:** Some data naturally comes as a never-ending stream of events. Stream processing **naturally fit with time series** data and detecting patterns over time.
- **Reason 2:** Batch processing lets the data build up and try to process them at once while stream processing process data as they come in hence spread the processing over time. Hence stream processing can work with **a lot less hardware than batch processing**.
- **Reason 3:** Sometimes data is huge and it is not even possible to store it. Stream processing let you handle large fire horse style data and **retain only useful bits**.

Why is stream processing need?



- **Reason 1:** Some data naturally comes as a never-ending stream of events. Stream processing **naturally fit with time series** data and detecting patterns over time.
- **Reason 2:** Batch processing lets the data build up and try to process them at once while stream processing process data as they come in hence spread the processing over time. Hence stream processing can work with **a lot less hardware than batch processing**.
- **Reason 3:** Sometimes data is huge and it is not even possible to store it. Stream processing let you handle large fire horse style data and **retain only useful bits**.
- **Reason 4:** Finally, there are **a lot of streaming data available**

Application Domains using Stream Processing?

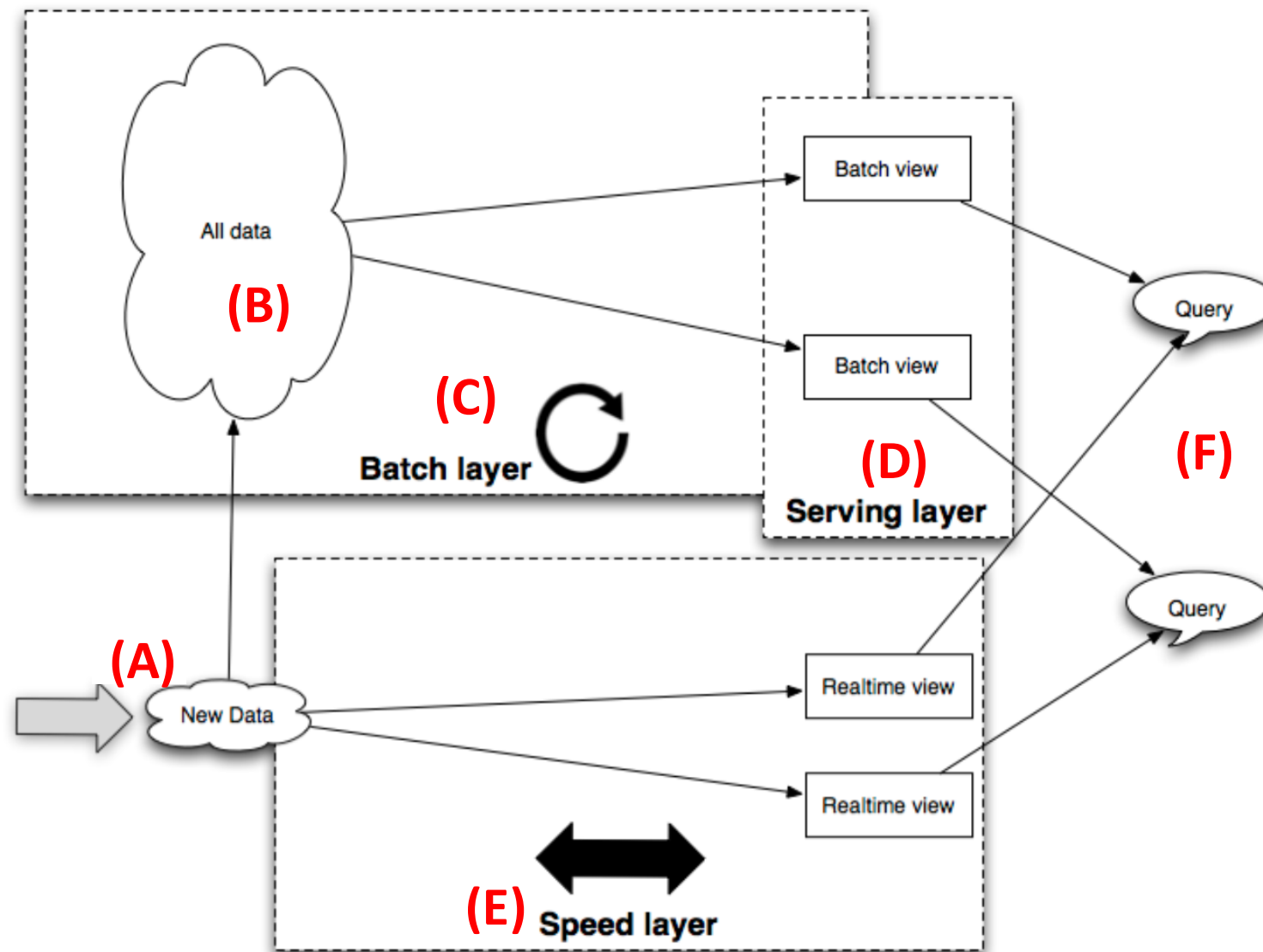


- Algorithmic Trading, Stock Market Surveillance,
- Smart Patient Monitoring
- Monitoring a production line
- Supply chain optimizations
- Intrusion, Surveillance and Fraud Detection (e.g. [Uber](#))
- Most Smart Device Applications: Smart Car, Smart Home ..
- Smart Grid — (e.g. load prediction and outlier plug detection see [Smart grids, 4 Billion events, throughout in range of 100Ks](#))
- Traffic Monitoring, Geofencing, Vehicle, and Wildlife tracking — e.g. [TFL London](#) Transport Management System
- Sports analytics — Augment Sports with real-time analytics (e.g. this is a work we did with a real football game (e.g. [Overlaying realtime analytics on Football Broadcasts](#)))
- Context-aware promotions and advertising
- Computer system and network monitoring
- Predictive Maintenance, (e.g. [Machine Learning Techniques for Predictive Maintenance](#))
- Geospatial data processing

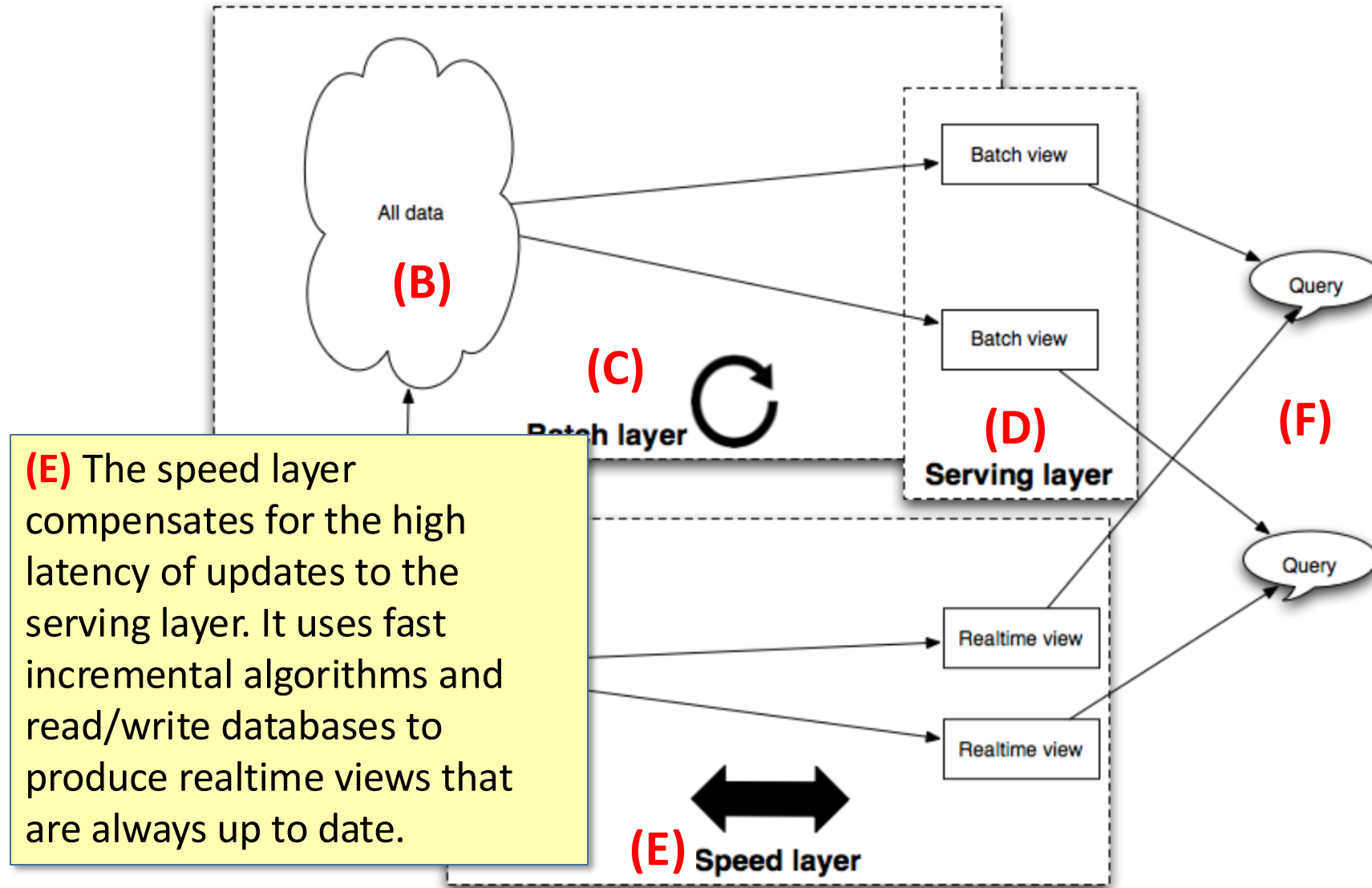


Lambda Architecture: Speed Layer

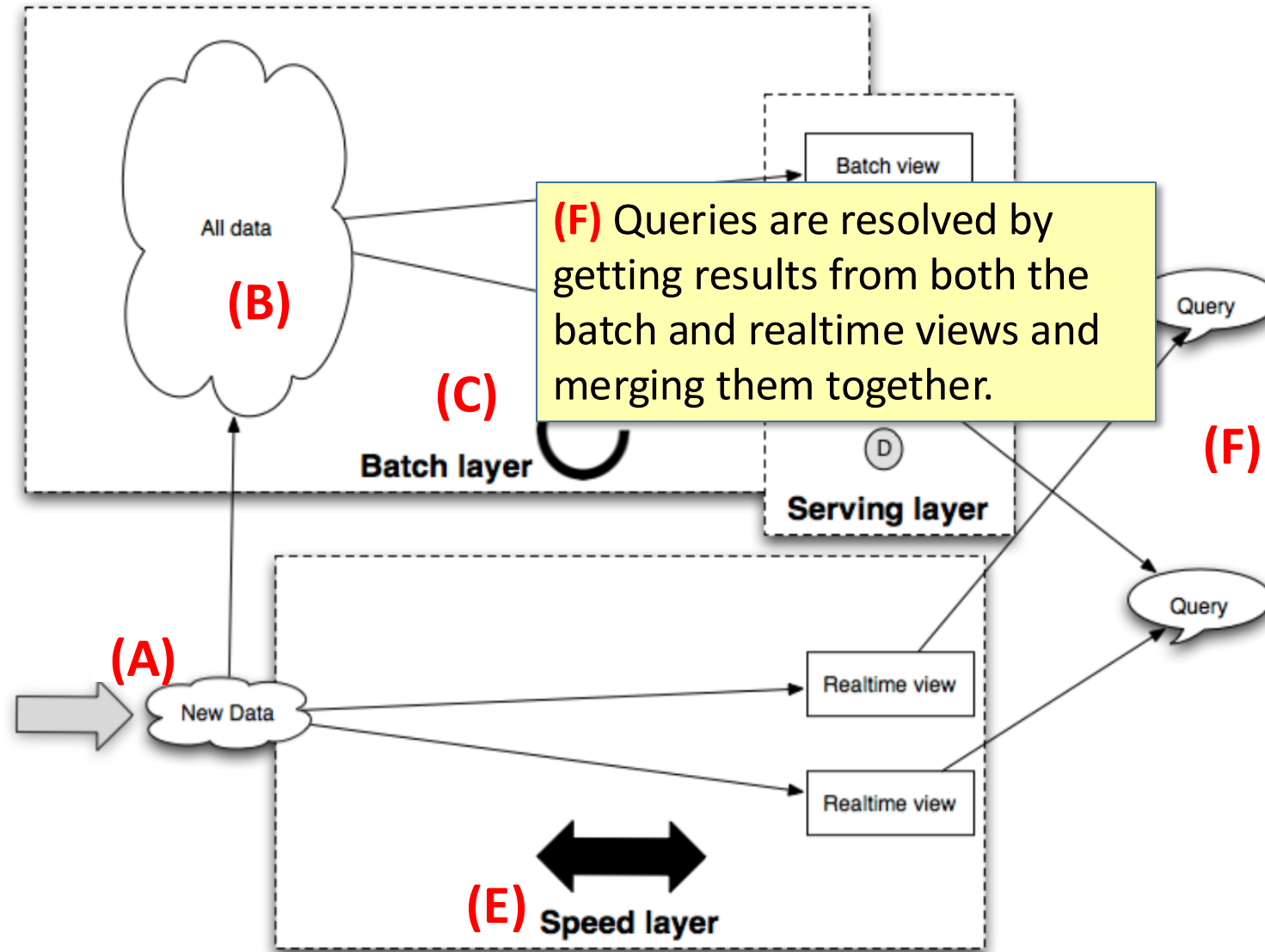
Lambda Architecture



Stream Processing in the Lambda Architecture



Stream Processing in the Lambda Architecture



Batch vs. Stream Processing



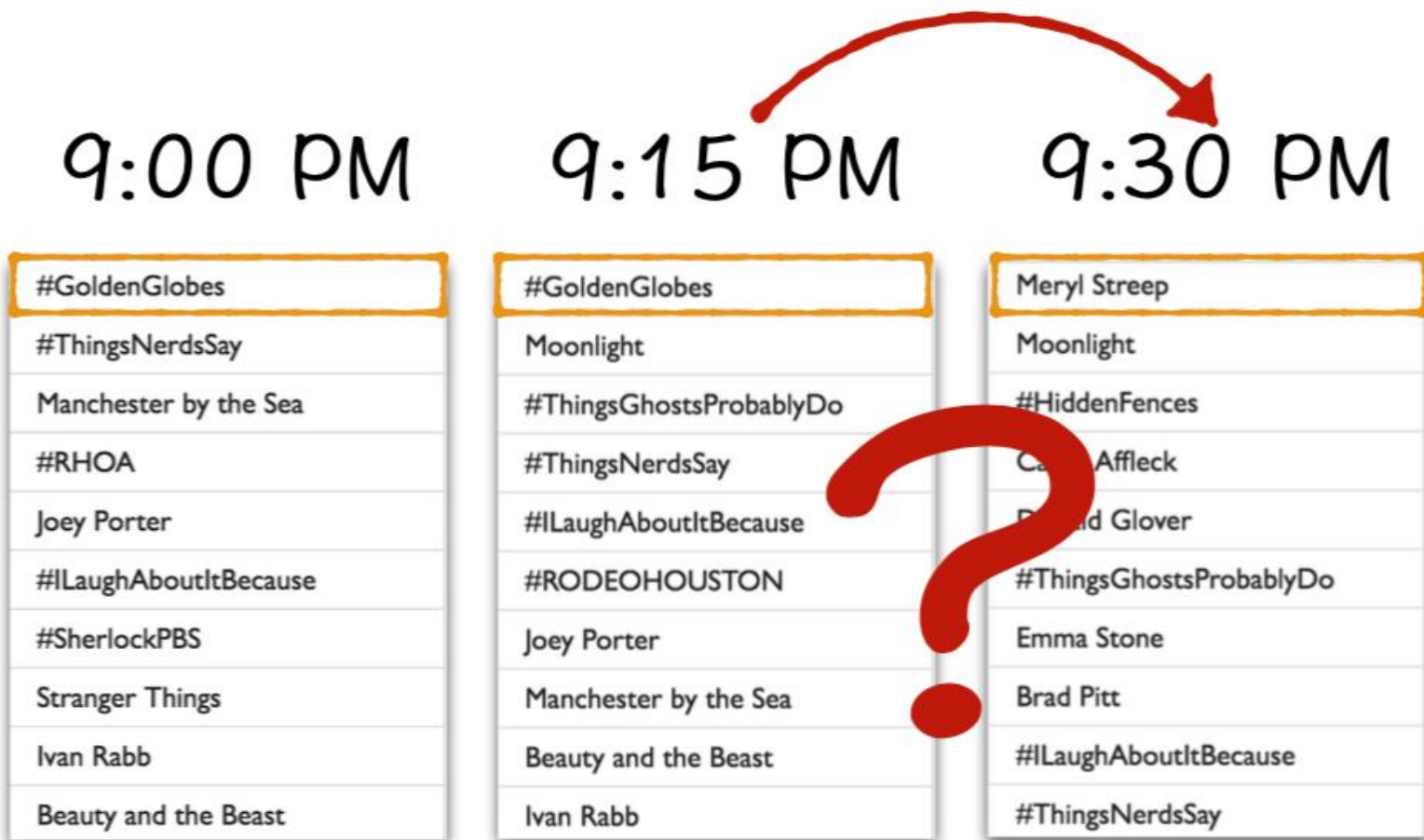
- Batch processing

- Gathering of data and processing as a group at one time.
- Jobs run to completion
- Data might be out of date

- Real-time processing

- Processing of data that takes place as the information is being entered.
- Run for ever

Batch vs Stream Processing



Batch vs Stream Processing



9:20 PM

9:21-9:29 PM



"Take your broken heart, make it into art" -- [#CarrieFisher](#) via [#MerylStreep](#)

find someone who looks at you the way everyone looks at **meryl streep**

Can't stop watching this lady having NONE of **Meryl Streep**. 😂😂😂
[#GoldenGlobes](#) 🏆

Meryl Streep's speech at the [#GoldenGlobes](#) 🏆 is definitely one of the biggest highlights of the night.

Celebrities couldn't contain their love for **Meryl Streep** after [#GoldenGlobes](#) 🏆
speech huff.to/2j8aj3Z

Goodbye [#MerylStreep](#) you just lost 60 million fans. Get use to movie flops.

Batch vs Stream Processing



9:21-9:29 PM

"Take your broken heart, make it into art" -- [#CarrieFisher](#) via [#MerylStreep](#)

find someone who looks at you the way everyone looks at meryl streep

Can't stop watching this lady having NONE of Meryl Streep. 😂😂😂
[#GoldenGlobes](#) 🏆

Meryl Streep's speech at the [#GoldenGlobes](#) 🏆 is definitely one of the biggest highlights of the night.

Celebrities couldn't contain their love for Meryl Streep after [#GoldenGlobes](#) 🏆 speech huff.to/2j8aj3Z

Goodbye [#MerylStreep](#) you just lost 60 million fans. Get use to movie flops.



9:30 PM

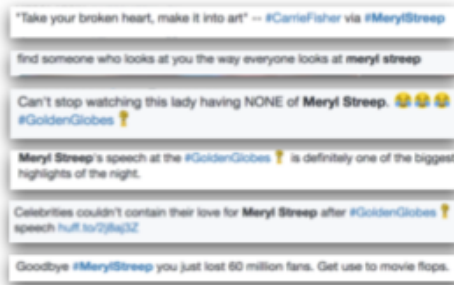
Meryl Streep
Moonlight
#HiddenFences
Casey Affleck
Donald Glover
#ThingsGhostsProbablyDo
Emma Stone
Brad Pitt
#ILaughAboutItBecause
#ThingsNerdsSay

Top keywords
in last 10 mins

Batch vs Stream Processing



9:21-9:29 PM



App

Batch processing

Process all tweets in the last 10 mins at a time

Stream processing

Process each tweet as it comes in

Batch vs Stream Processing



- Start at 9:00 PM
- Collect tweets for 10 mins
- Process tweets for 2 mins
- First trends at 9:12 PM
- Next update at 9:24 PM

Batch processing

Process all tweets in the last 10 mins at a time

Batch vs Stream Processing



- Start at 9:00 PM
- Collect tweets for 10 mins
- First trends at 9:10 PM
- As each tweet comes in, trends are updated

Stream processing
Process each tweet
as it comes in

Batch vs Stream Processing

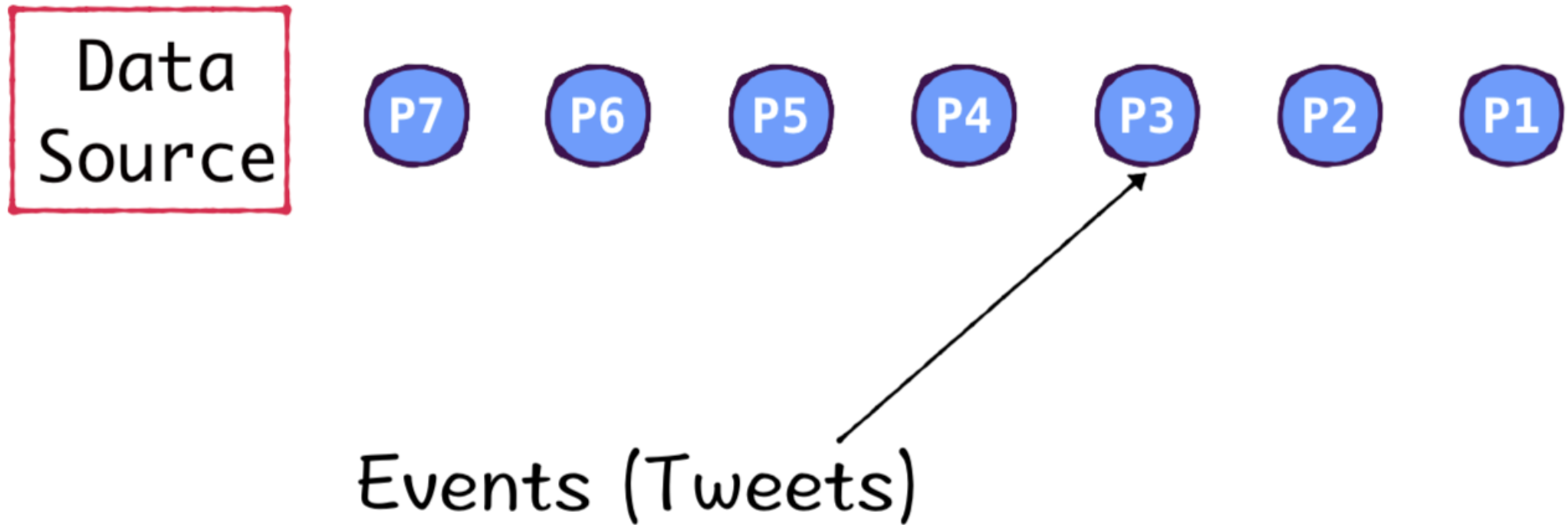


Stream processing

Get the most recent (real-time) view

React to events immediately

Batch vs Stream Processing



Batch vs Stream Processing



Batch vs Stream Processing



Gap increased during processing time



Batch vs Stream Processing



If processing time $\gg$ event arrival rate



Batch vs Stream Processing



How do we reduce processing time?



How to do Stream Processing?



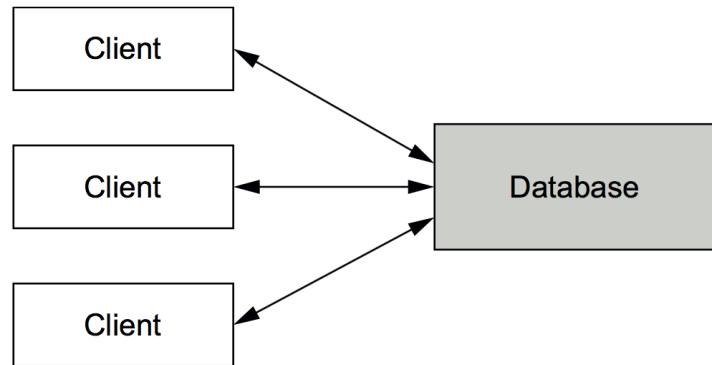
- Queue Message Management Systems

Queuing



Synchronous

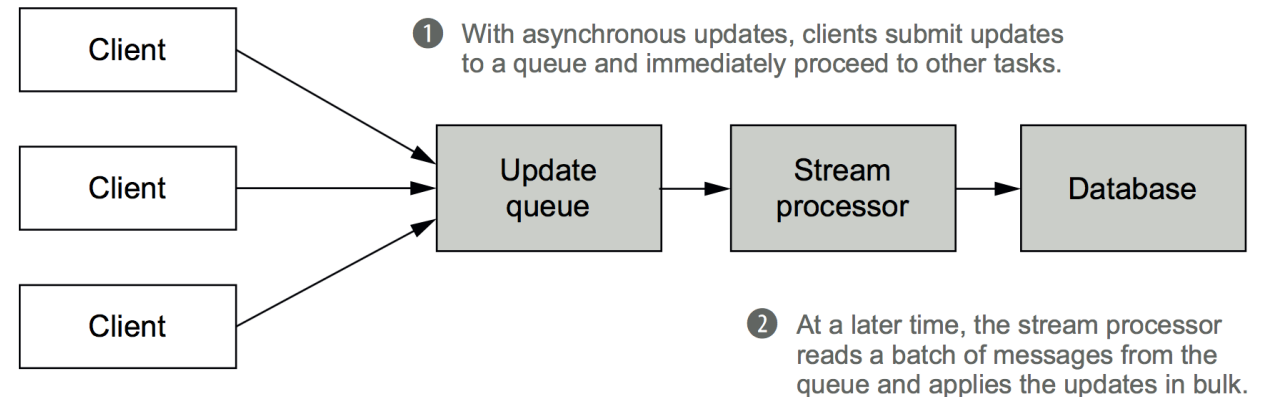
Near-transactional systems that interact with users and require coordination with the user interface.



VS

Asynchronous

Common for analytics-oriented workloads or workloads not requiring coordination.



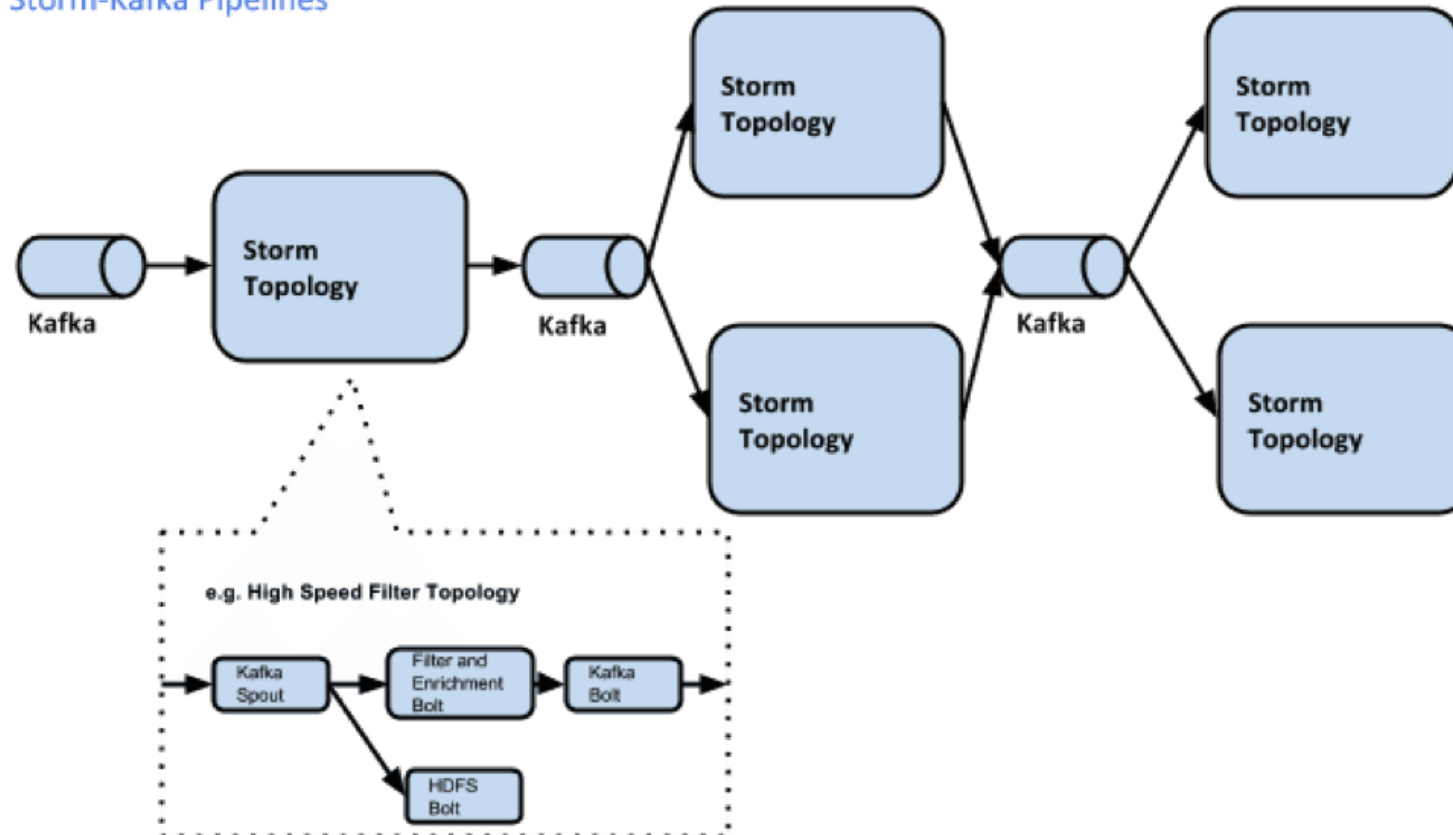
- Single-consumer queue server
 - The event is not immediately removed, **Only when an event is acked it will be removed from the queue.**
 - Queue implementations such as **Kestrel** and **RabbitMQ** use a similar interface
- Multi-consumer queues
 - Shift the responsibility of tracking the consumed/unconsumed status of events from the queue to the applications themselves
 - Apache **Kafka** is a good example of a queue implementation that implements multi-consumer queuing.

How to do Stream Processing?



- Queue Message Management Systems
- Stream Processing Frameworks

Storm-Kafka Pipelines



- Plethora of emerging technologies 4 Stream Processing
 - <https://thenewstack.io/apache-streaming-projects-exploratory-guide/>
- Apache Spark Streaming
- Apache Storm
- Apache Kafka Stream
- Apache Flink
- Apache Flume
- Apache S4
- Apache NiFi
- ...

Stream Processing - Desiderata



- Exactly-once guarantees: state in statefull operators should be correctly restored after a failure
- Low latency: the lower the better. Many applications require sub-second latency
- High throughput: pushing large amounts of data through the pipeline is crucial as the data rates grow
- Powerful computation model: the framework should offer a programming model that does not restrict the user and allows a wide variety of applications
- Low overhead of the fault tolerance mechanism in the absence of failures
- Flow control: backpressure from slow operators should be naturally absorbed by the system and the data sources to avoid crashes or degrading performance due to slow consumers
- <https://www.ververica.com/blog/high-throughput-low-latency-and-exactly-once-stream-processing-with-apache-flink>

Stream Processing



- Micro-batched
 - Spark streaming
 - Trident
- One-at-a-time
 - Apache Storm
 - Apache Kafka
 - Alerting
 - Financial trading
 - ...

	One-at-a-time	Micro-batched
Lower latency	✓	
Higher throughput		✓
At-least-once semantics	✓	✓
Exactly-once semantics	In some cases	✓
Simpler programming model	✓	

Micro-batching vs One-at-a-time



Micro-batch processing is the practice of collecting data in small groups (“batches”) for the purposes of taking action on (processing) that data. Contrast this to traditional “batch processing,” which often implies taking action on a large group of data. Micro-batch processing is a variant of traditional batch processing in that the data processing occurs more frequently so that smaller groups of new data are processed. In both micro-batch processing and traditional batch processing, data is collected based on a predetermined threshold or frequency before any processing occurs.

Micro-batching vs One-at-a-time



- Micro-batch processing accelerated the cycle so data could be loaded much more frequently, sometimes in increments as small as seconds. Micro-batch loading technologies include Fluentd, Logstash, and Apache Spark Streaming.
- Micro-batch processing is very similar to traditional batch processing in that data are usually processed as a group. The primary difference is that the batches are smaller and processed more often. A micro-batch may process data based on some frequency – for example, you could load all new data every two minutes (or two seconds, depending on the processing horsepower available). Or a micro-batch may process data based on some event flag or trigger (the data is greater than 2 megabytes or contains more than a dozen pieces of data, for example).
- Though it is not true real-time processing, micro-batch processing initially sufficed to support some “real-time” use cases in which data does not need to be up-to-the-moment accurate. For example, you may have corporate dashboards that refresh every 15 minutes or hourly. Or you may collect server logs in small regular intervals for historical record-keeping rather than for true real-time use cases such as intrusion detection.

Micro-batching vs One-at-a-time



- The world has accelerated, and there are many use cases for which micro-batch processing is simply not fast enough. Organizations now typically only use micro-batch processing in their applications if they have made architectural decisions that preclude stream processing. For example, an Apache Spark shop may use Spark Streaming, which is – despite its name and use of [in-memory compute](#) resources – actually a micro-batch processing extension of the Spark API.
- Now, [stream processing](#) technologies are becoming the go-to for modern applications. As data has accelerated throughout the past decade, enterprises have turned to real-time processing to respond to data closer to the time at which it is created to solve for a variety of use cases and applications. In some cases, micro-batch processing was “real-time enough,” but increasingly organizations have recognized that stream processing that leverages in-memory technology – either in the cloud or on-premises – is the ideal solution.
- Stream processing allows applications to respond to new data events at the moment they are created. Rather than grouping data and collecting it at some predetermined interval, stream processing systems collect and process data immediately as they are generated.

Messages delivery guarantees



- **At-most once Configuration**

- As the name suggests, At-most-once means the message will be delivered at-most once. Once delivered, there is no chance of delivering again. If the consumer is unable to handle the message due to some exception, the message is lost.

- **At-least once configuration**

- At-least once as the name suggests, message will be delivered atleast once. There is high chance that message will be delivered again as duplicate.

- **Exactly-once configuration**

- Exactly-once as the name suggests, there will be only one and once message delivery. It difficult to achieve in practice.

Messages delivery guarantees



- **At-most once is the cheapest**—highest performance, least implementation overhead—because it can be done in a fire-and-forget fashion without keeping state at the sending end or in the transport mechanism.
- **At-least once requires retries to counter transport losses**, which means keeping state at the sending end and having an acknowledgement mechanism at the receiving end.
- **Exactly once is the most expensive**—and has consequently worst performance—because in addition to the second it requires state to be kept at the receiving end in order to filter out duplicate deliveries

How to do Stream Processing?



- Queue Message Management Systems
- Stream Processing Frameworks

Since 2016, a new idea called Streaming SQL

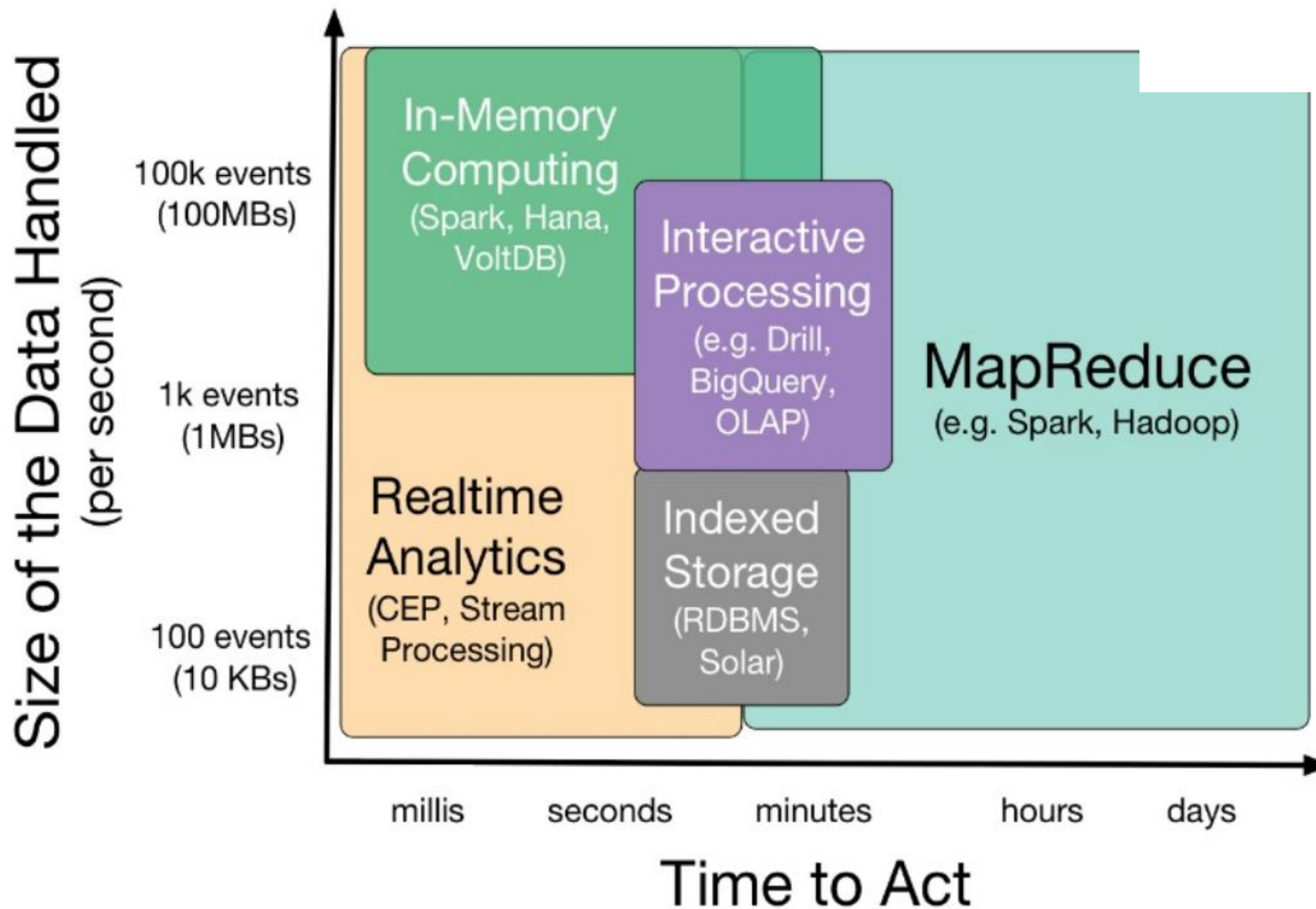
Writing Streaming Applications requires different thinking patterns from writing code with a language like Java or Python.

Where to store data?



- In memory:
 - Stream and Table
 - DataFrame
- Secondary storage:
 - File System
 - Queue
 - DBMS
 - NoSQL DB

Landascape



Desiderata



	Record acks (Storm)	Micro-batching (Spark Streaming, Trident)	Transactional updates (Google Cloud Dataflow)	Distributed snapshots (Flink)
Guarantee	At least once	Exactly once	Exactly once	Exactly once
Latency	Very Low	High	Low (delay of transaction)	Very Low
Throughput	Low	High	Medium to High (Depends on throughput of distributed transactional store)	High
Computation model	Streaming	Micro-batch	Streaming	Streaming
Overhead of fault tolerance mechanism	High	Low	Depends on throughput of distributed transactional store	Low
Flow control	Problematic	Problematic	Natural	Natural
Separation of application logic from fault tolerance	Partially (timeouts matter)	No (micro batch size affects semantics)	Yes	Yes



Apache Kafka

- In Big Data, an enormous volume of data is used. There are two main challenges:
 1. to collect large volume of data;
 2. to analyze the collected data.
- To overcome those challenges, we must need a messaging system.
- **Kafka** is designed for distributed high throughput message management system.
- Kafka tends to work very well as a replacement for a more traditional message broker.

What is a Messaging System?



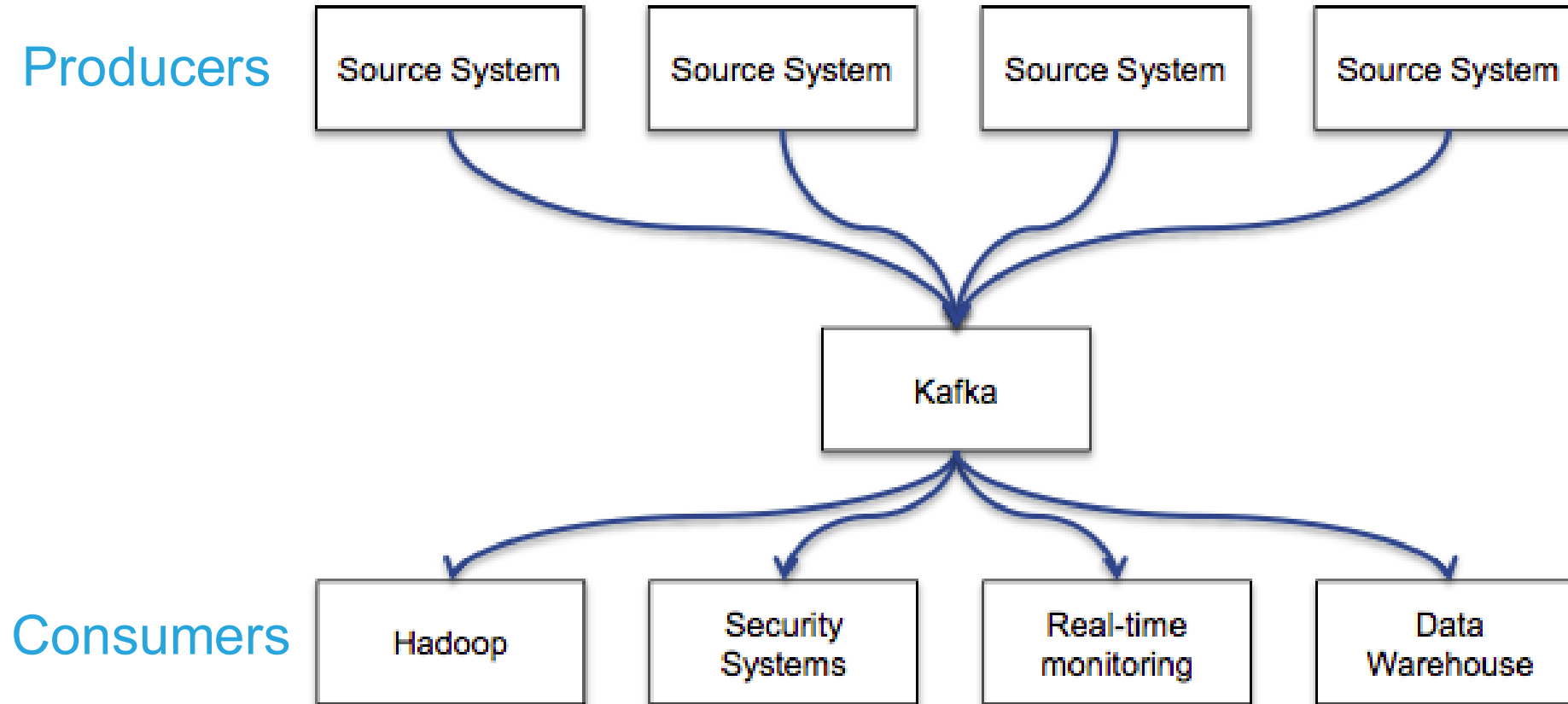
- A Messaging System is responsible for transferring data from one application to another, so the applications can focus on data, but not worry about how to share it.
- Distributed messaging is based on the concept of reliable message queuing.
- Messages are queued asynchronously between client applications and messaging system.

Kafka vs traditional message brokers



- One of the major things that sets Kafka apart from “traditional” message brokers like RabbitMQ or ActiveMQ, is that a topic in Kafka doesn’t know or care about its consumers. It’s simply a log that consumers can dive into and access data from, at any time.
- On the other hand, on a traditional message broker, messages are delivered only once to consumers. And in traditional topics, consumers who subscribe to a topic can only receive messages from that point forward, they can’t rewind.

Why Kafka



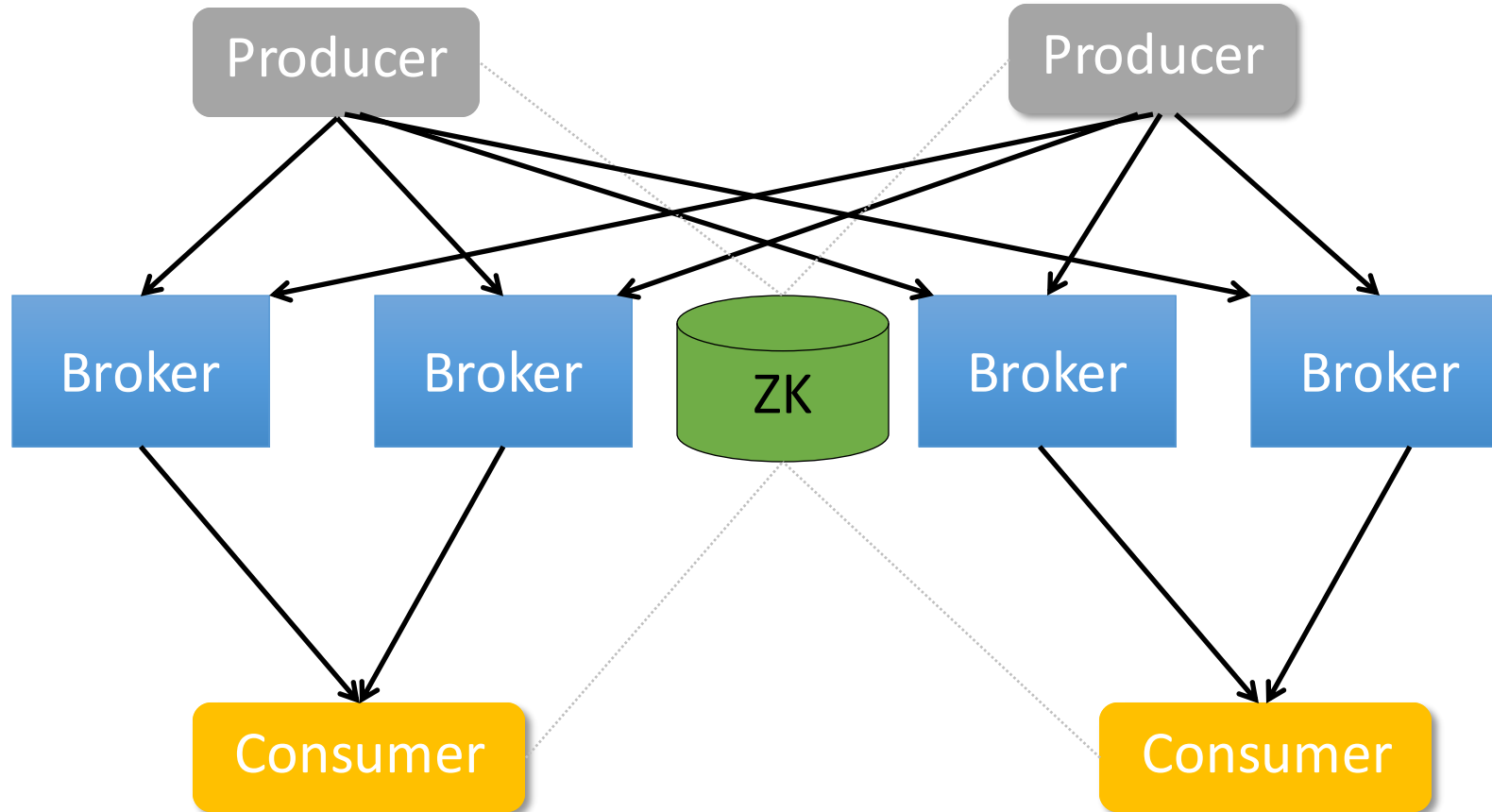
Kafka decouples Data Pipelines

Key Terminology



- Kafka maintains feeds of messages in categories called **topics**.
- Processes that publish messages to a Kafka topic are called **producers**.
- Processes that subscribe to topics and process the feed of published messages are called **consumers**.
- Kafka runs as a cluster comprised of one or more servers each of which is called a **broker**.
- Communication between all components is done via a high performance simple API over TCP protocol

Architecture



- Message Ordering
 - Ordering is only guaranteed within a partition for a topic
 - To ensure ordering:
 - ❖ Group messages in a partition by key (producer)
 - ❖ Configure exactly one consumer instance per partition within a consumer group
- Guarantees
 - Messages sent by a producer to a particular topic partition will be appended in the order they are sent
 - A consumer instance sees messages in the order they are stored in the log
 - For a topic with replication factor N , Kafka can tolerate up to $N-1$ server failures without “losing” any messages committed to the log

Topics - Replication



- Topics can (and should) be replicated.
- The unit of replication is the partition
- Each partition in a topic has 1 leader and 0 or more replicas.

Producer

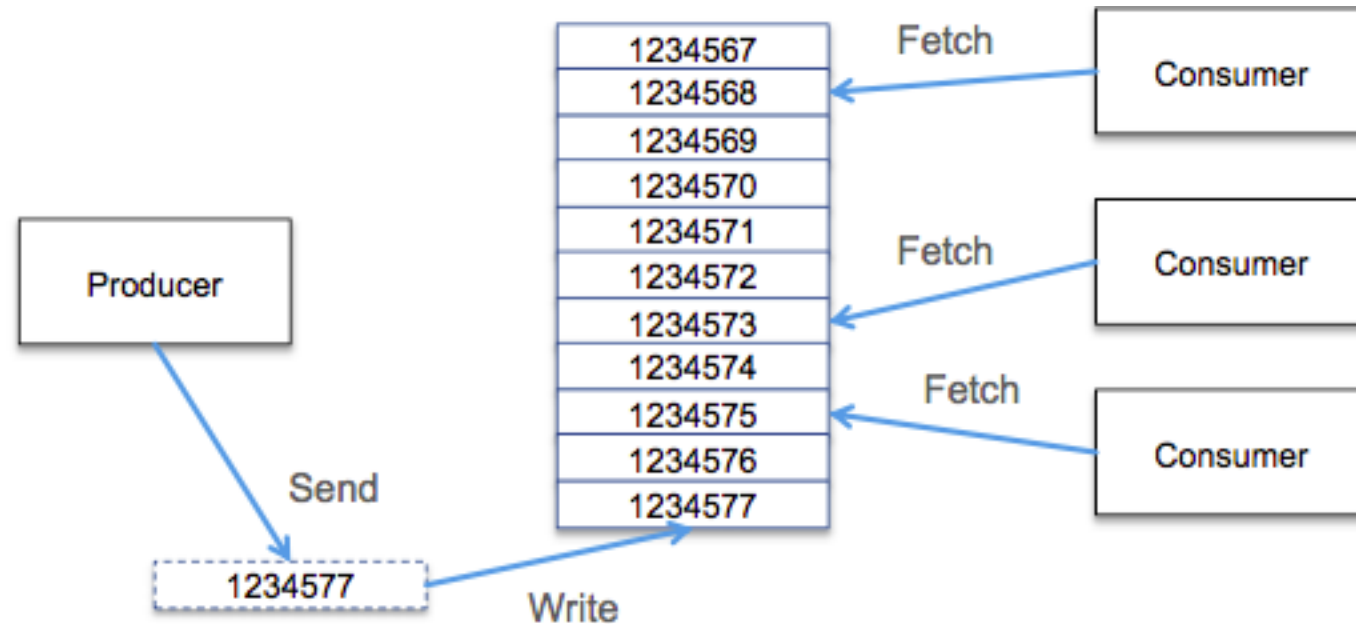


- Producers publish to a topic of their choosing (push)
- Load can be distributed
 - Typically by “round-robin”
 - Can also do “semantic partitioning” based on a key in the message
- Brokers load balance by partition
- Can support async sending
- All nodes can answer metadata requests about:
 - Which servers are alive
 - Where leaders are for the partitions of a topic

Consumer



- Multiple Consumers can read from the same topic
- Each Consumer is responsible for managing it's own offset
- Messages stay on Kafka...they are not removed after they are consumed

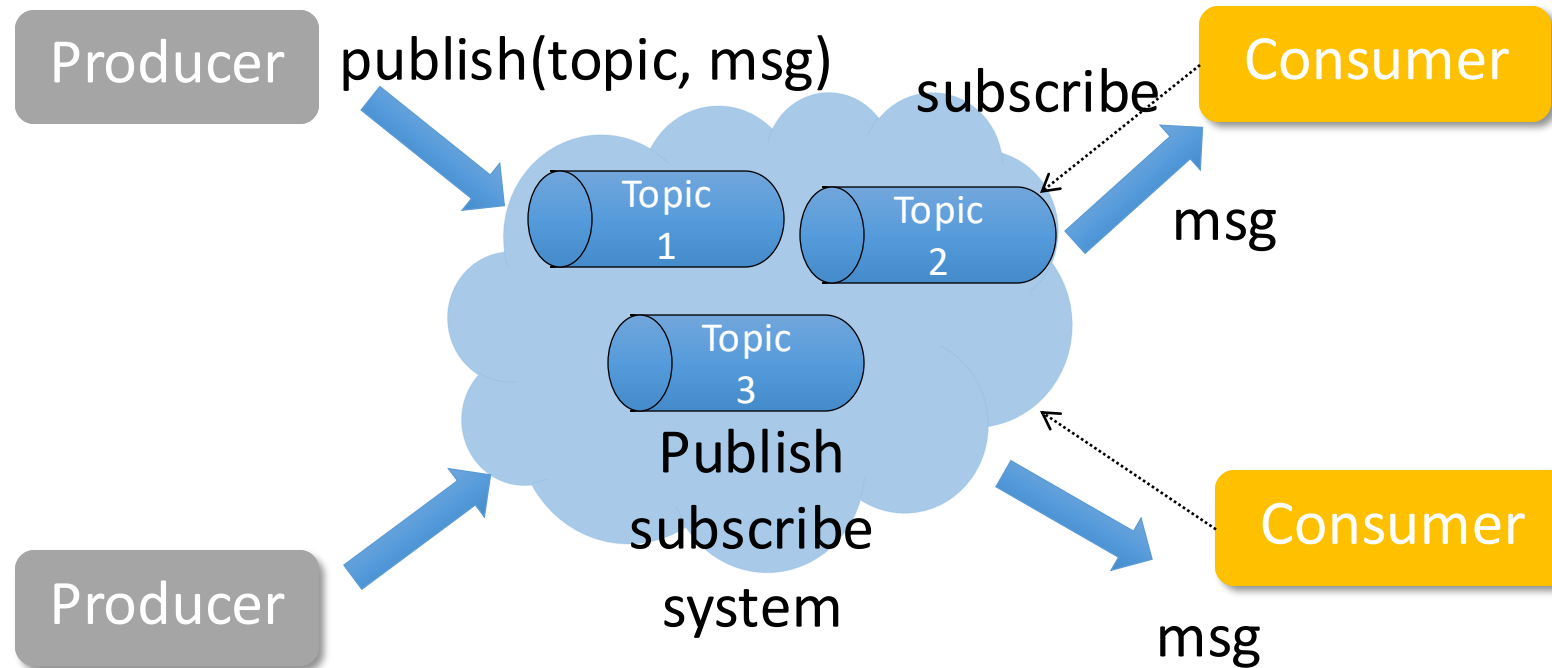


Publish/Subscribe System



- Apache Kafka is a distributed publish-subscribe messaging system and a robust queue that can handle a high volume of data and enables you to pass messages from one end-point to another.
- Kafka is suitable for both offline and online message consumption.
- Kafka messages are persisted on the disk and replicated within the cluster to prevent data loss.
- Kafka is built on top of the ZooKeeper synchronization service.
- It integrates very well with Apache Storm and Spark for real-time streaming data analysis.

Publish/Subscribe System



Role of ZooKeeper



- ZooKeeper is a distributed configuration and synchronization service.
- Zookeeper serves as the coordination interface between the Kafka brokers and consumers.
- The Kafka servers share information via a Zookeeper cluster. Kafka stores basic metadata in Zookeeper such as information about topics, brokers, consumer offsets (queue readers) and so on.

Benefits



- **Reliability** - Kafka is distributed, partitioned, replicated and fault tolerance.
- **Scalability** - Kafka messaging system scales easily without down time.
- **Durability** - Kafka uses “Distributed commit log” which means messages persists on disk as fast as possible, hence it is durable.
- **Performance** - Kafka has high throughput for both publishing and subscribing messages. It maintains stable performance even many TB of messages are stored.
- Kafka is very fast and guarantees zero downtime and zero data loss.

Accessing Kafka in Python



- There are multiple Python libraries available for usage:
 - [Kafka-Python](#) – An open-source community-based library.
 - [PyKafka](#) – This library is maintained by Parsly and it's claimed to be a Pythonic API.
Unlike Kafka-Python you can't create dynamic topics.
 - [Confluent Python Kafka](#):- It is offered by Confluent as a thin wrapper around librdkafka, hence it's performance is better than the two.
- For this post, we will be using the open-source *Kafka-Python*.
- Give an outlook [here](#).

Kafka usage (docker-compose)



- <https://gist.github.com/sahilsk/d2a6ec384f5f2333e3fef40a581a97e1>
- <https://docs.confluent.io/platform/current/platform-quickstart.html#step-2-create-ak-topics-for-storing-your-data>



Realtime Streaming Analytics Patterns

- **1: Preprocessing**

- Preprocessing is often done as a projection from one data stream to the other or through filtering. Potential operations include
 - ❖ Filtering and removing some events
 - ❖ Reshaping a stream by removing, renaming, or adding new attributes to a stream
 - ❖ Splitting and combining attributes in a stream
 - ❖ Transforming attributes
- For example, from a twitter data stream, we might choose to extract the fields: author, timestamp, location, and then filter them based on the location of the author.

- **2: Alerts and Thresholds**

- This pattern detects a condition and generates alerts based on a condition. (e.g. Alarm on high temperature). These alerts can be based on a simple value or more complex conditions such as rate of increase etc.
 - ❖ In TFL (Transport for London) [demo video](#) based on transit data from London, we trigger a speed alert when the bus has exceeded a given speed limit.
 - ❖ Generate alerts for scenarios such as the server room temperature is continually increasing for last 5 mins.



• 3: Simple Counting and Counting with Windows

- This pattern includes aggregate functions like Min, Max, etc., and they can be counted without storing any data.
- Count are often useful with a time window:
 - Time
 - ❖ **Sliding window:** keeps each event for the given time window, produce an output whenever new event has added or removed.
 - ❖ **Batch window:** also called tumbling windows, they only produce output at the end of the time window
 - Length
 - ❖ **Sliding window:** same as the time, sliding window, but keeps an window of n events instead of selecting them by time.
 - ❖ **Batch window:** same as the time, batch window, but keeps an window of n events instead of selecting them by time.

- **4: Joining Event Streams**

- Main idea behind this pattern is to match up multiple data streams and create a new event stream.
- For an example, let's assume we play a football game with both the **players and the ball having sensors** that emit events with current location and acceleration.
- We can use “joins” to detect when a player have kicked the ball. To that end, we can join the ball location stream and the player stream on the condition that they are close to each other by one meter and the ball's acceleration has increased by more than fixed value.

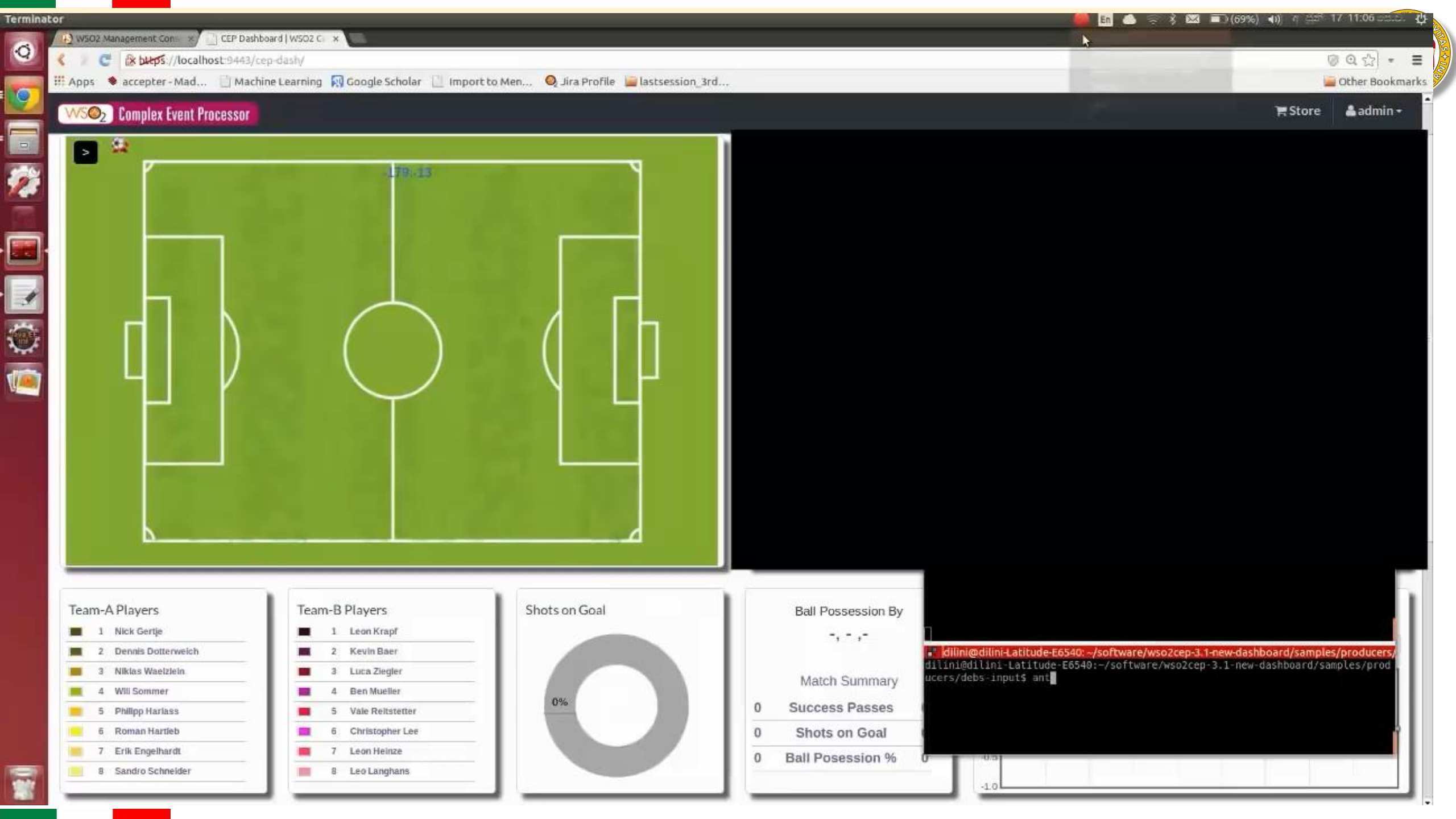
- **5: Data Correlation, Missing Events, and Erroneous Data**
 - Matching up two data streams that send events in different speeds
 - Detecting a missing event in a data stream (e.g. detect a customer request that has not been responded within in 1 hour of its reception.)
 - Detecting erroneous data (e.g. Detect failed sensors using a set of sensors that monitor overlapping regions and using those redundant data to find erroneous sensors and removing their data from further processing)

- **6: Interacting with Databases**

- When a transaction happened, lookup the age using the customer ID from customer database to be used for Fraud detection (enrichment)
- Checking a transaction against blacklists and whitelists in the database
- Receive an input from the user (e.g. Daily discount amount may be updated in the database, and then the query will pick it automatically without human intervention.)

- **7: Detecting Temporal Event Sequence Patterns**

- Using regular expressions with strings, we detect a pattern of characters from sequence of characters. Similarly, given a sequence of events, we can write a regular expression to detect a temporal sequence of events arranged on time where each event or condition about the event is parallel to a character in a string in the above example.
- [Demo video](#)



• 8: Tracking

- Tracking a fleet of vehicles, making sure that they adhere to speed limits, routes, and geo-fences.
- Tracking wildlife, making sure they are alive (they will not move if they are dead) and making sure they will not go out of the reservation.
- Tracking airline luggages and making sure they are not been sent to wrong destinations
- Tracking a logistic network and figure out bottlenecks and unexpected conditions.

- **9: Detecting Trends**

- We often encounter time series data. Detecting patterns from time series data and bringing them into operator attention are common use cases (Stock markets and Algorithmic trading, Predictive maintenance).
 - ❖ Rise, Fall
 - ❖ Turn (switch from rise to a fall)
 - ❖ Outliers
 - ❖ Complex trends

Realtime Streaming Analytics Patterns



- **10: Running the same Query in Batch and Realtime Pipelines**
- **11: Detecting and switching to Detailed Analysis**
- **12: Using a Model.**
- **13: Online Control**

Realtime Streaming Analytics Patterns



- Some more patterns are described here

<https://github.com/suhothayan/DEBS-2015-Realtime-Analytics-Patterns>

What is Machine Learning from Streaming Data?



- A model that takes into account **recent history** when it makes its predictions. A good example is the weather; If it has been sunny and 80 degrees the last two days, it is unlikely that it will be 20 and snowing the next day.
 - The prediction conditioned on history, **time-series prediction**.
- A model that is **updatable**. That is, the model in some sense “evolve” as data streams through their infrastructure. A good example might be a retail sales model that remains accurate as the business gets larger.
 - To update the model or retrain completely, is about dealing with **non-stationarity**

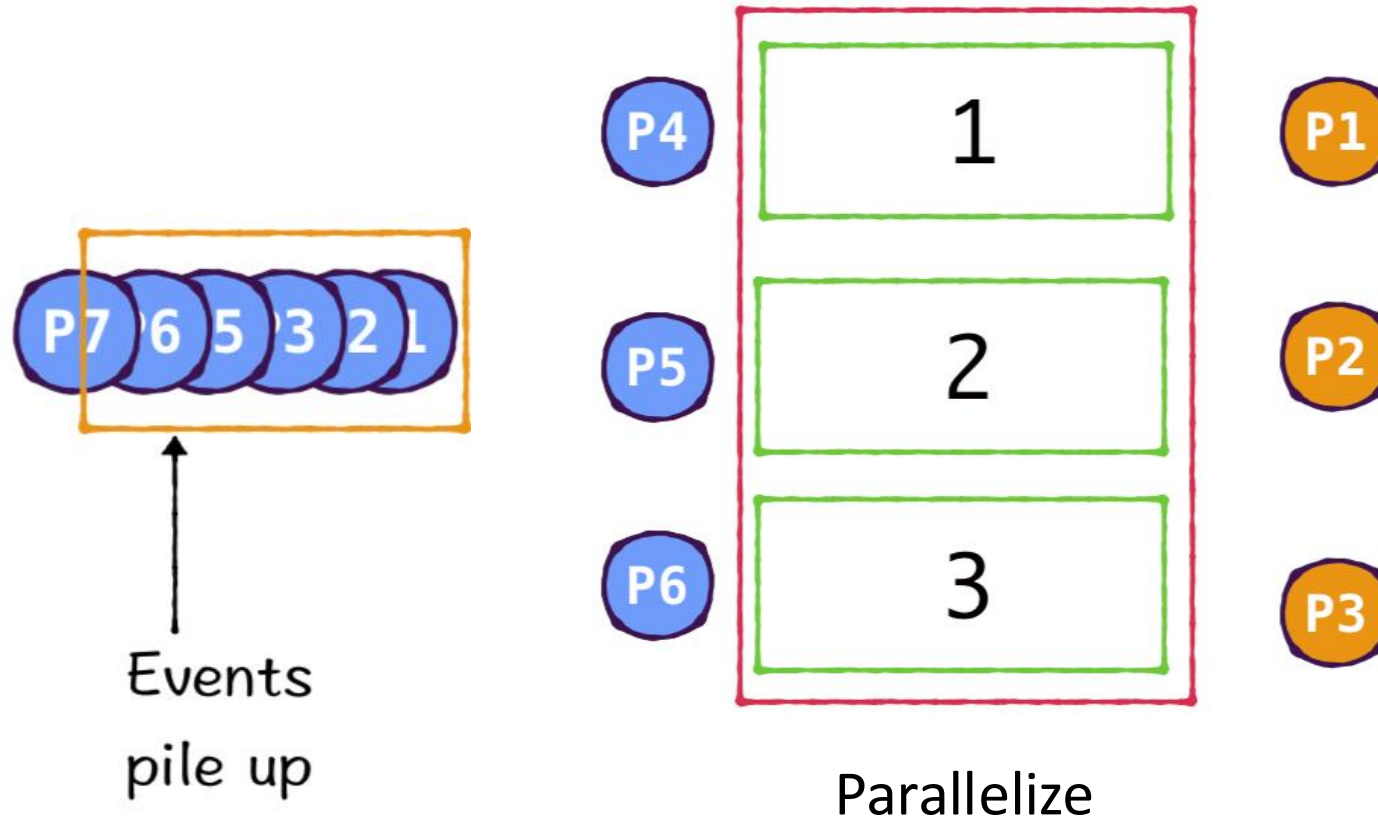


Apache Storm

Batch vs Stream Processing



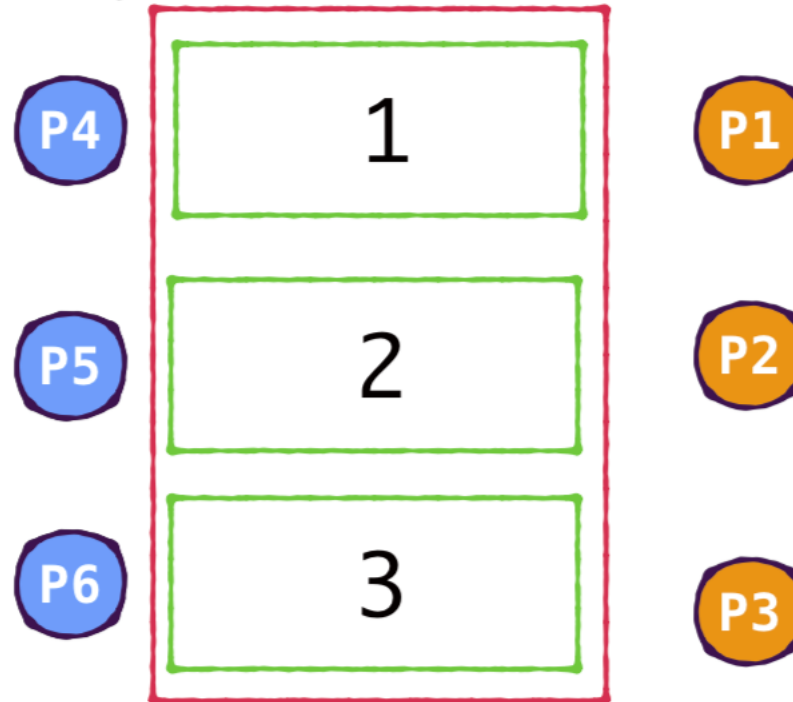
Distribute the stream to multiple components



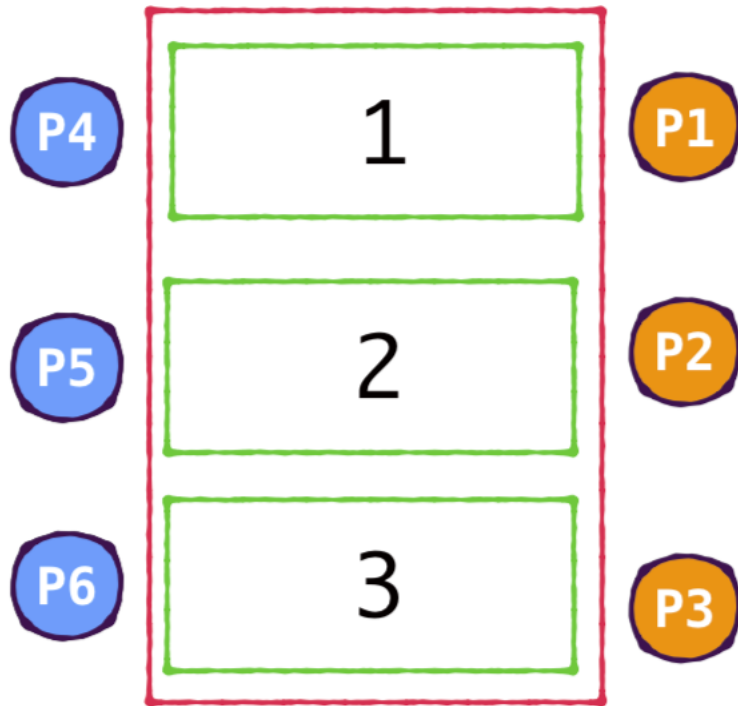
Batch vs Stream Processing



Apache Storm



Batch vs Stream Processing



Apache Storm

A framework for
distributed
stream processing



Apache Storm



- Apache Storm is a distributed real-time big data-processing system.
- Storm is designed to process vast amount of data in a fault-tolerant and horizontal scalable method.

One-at-a-time Paradigm – Storm Model



- Topologies, a Storm **topology** is analogous to a MapReduce job. One key difference is that a MapReduce job finishes.
- A **spout** is a source of streams in a topology. Generally spouts will read tuples from an external source and emit them into the topology (e.g. a Kestrel queue or the Twitter API).
- All processing in topologies is done in **bolts**. Bolts can do anything from filtering, functions, aggregations, joins, talking to databases, and more.

Benefits

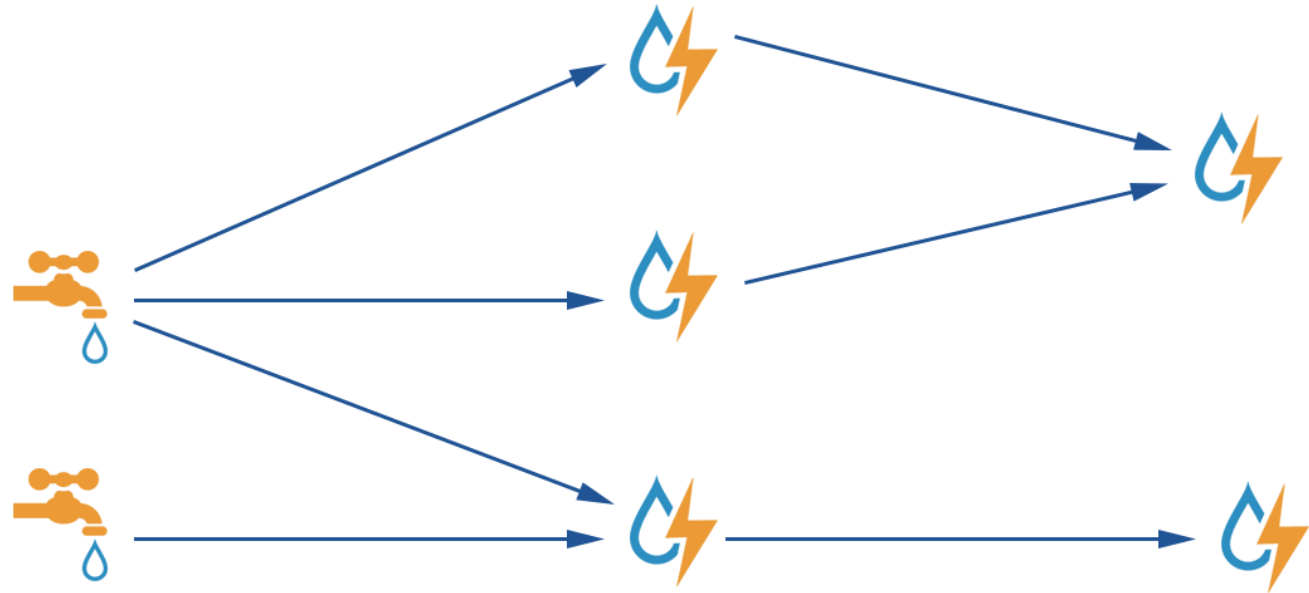


- Storm is open source, robust, and user friendly.
- Storm is fault tolerant, flexible, reliable, and supports any programming language.
- Storm is unbelievably fast because.
- Storm is highly scalable.
- Storm has very low latency.
- Storm has operational intelligence.
- Storm provides guaranteed data processing even if any of the connected nodes in the cluster die or messages are lost.

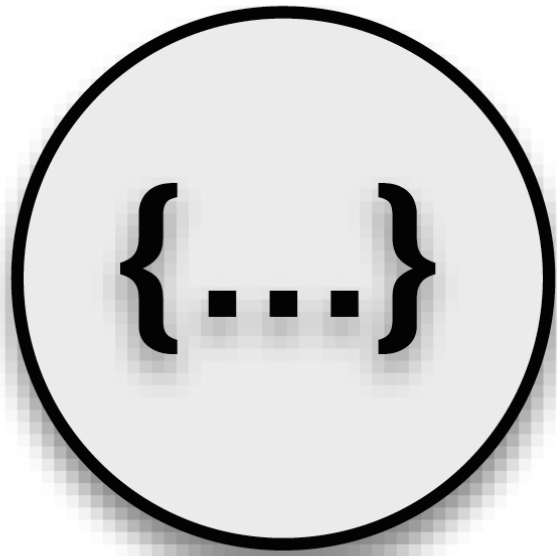
Components



- Topologies
- Streams
- Spouts
- Bolts
- Stream groupings
- Reliability
- Tasks
- Workers

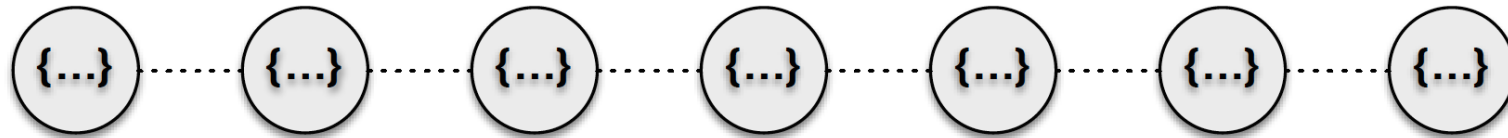


Tuple



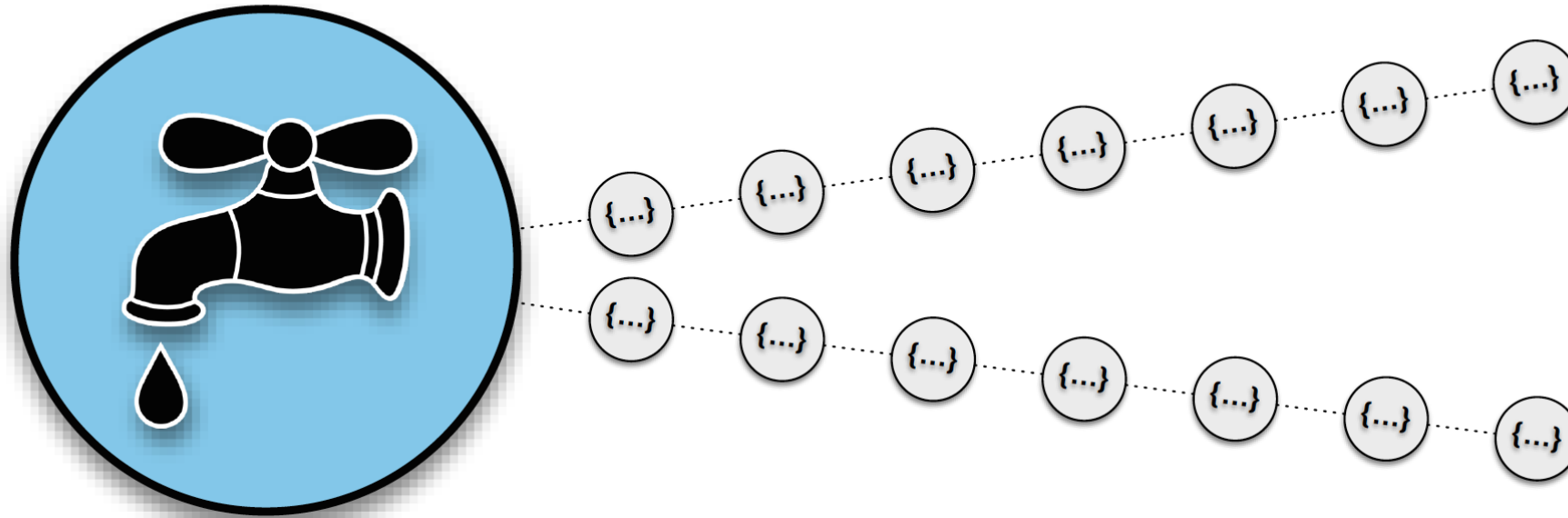
- Core Unit of Data
- Immutable Set of Key/Value Pairs

Streams



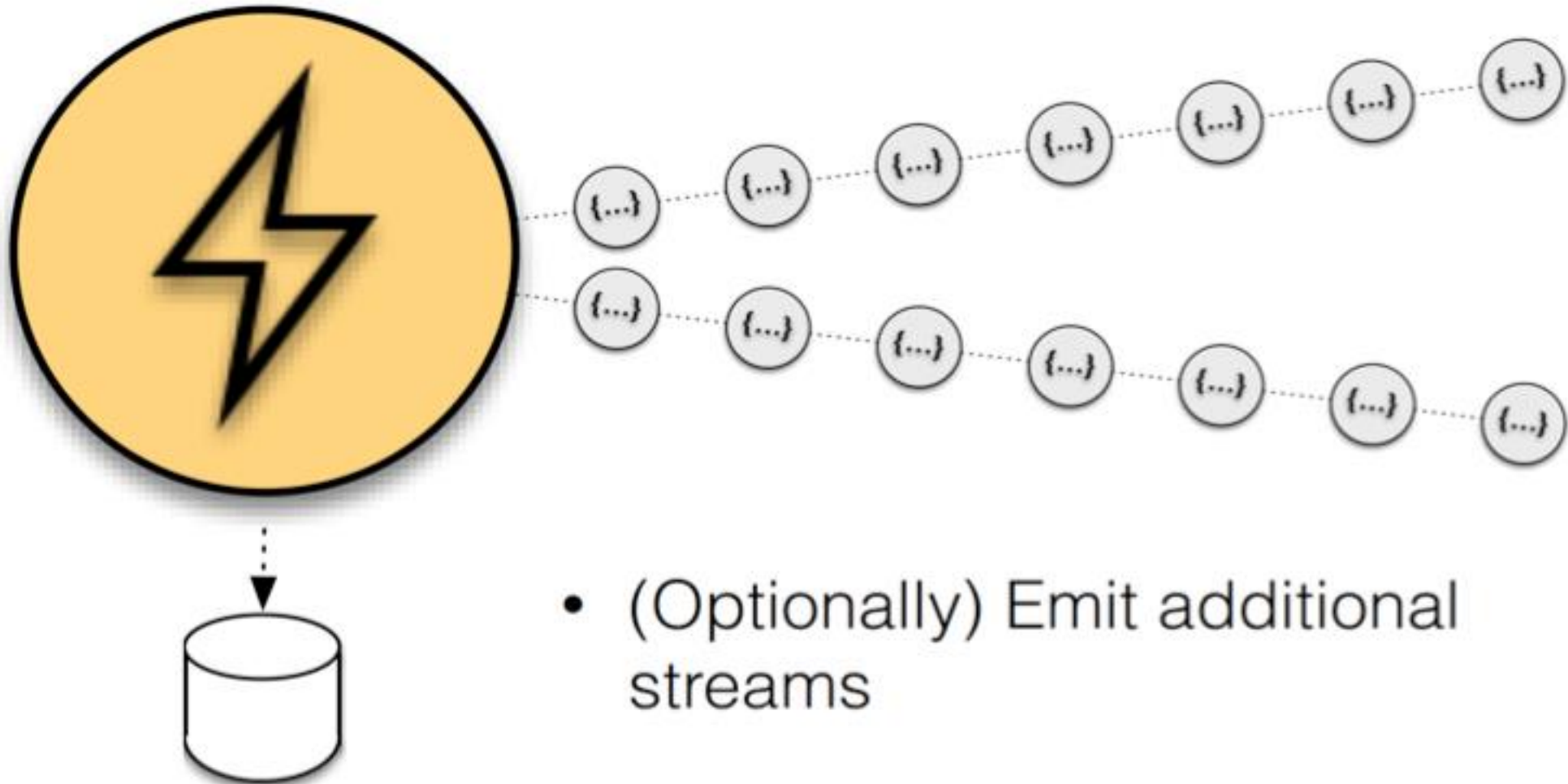
Unbounded Sequence of Tuples

Spouts



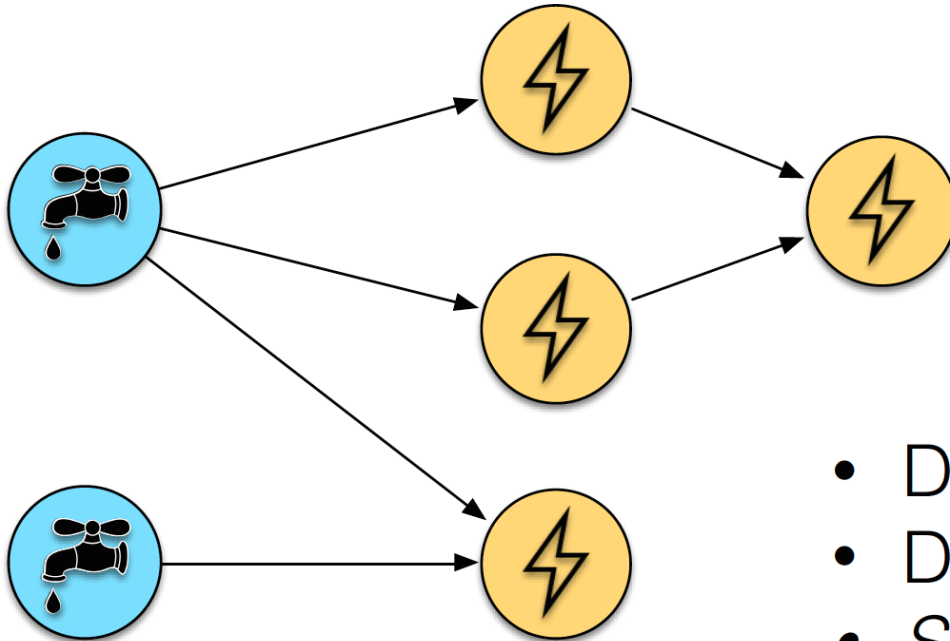
- Source of Streams
- Wraps a streaming data source and emits Tuples

Bolts



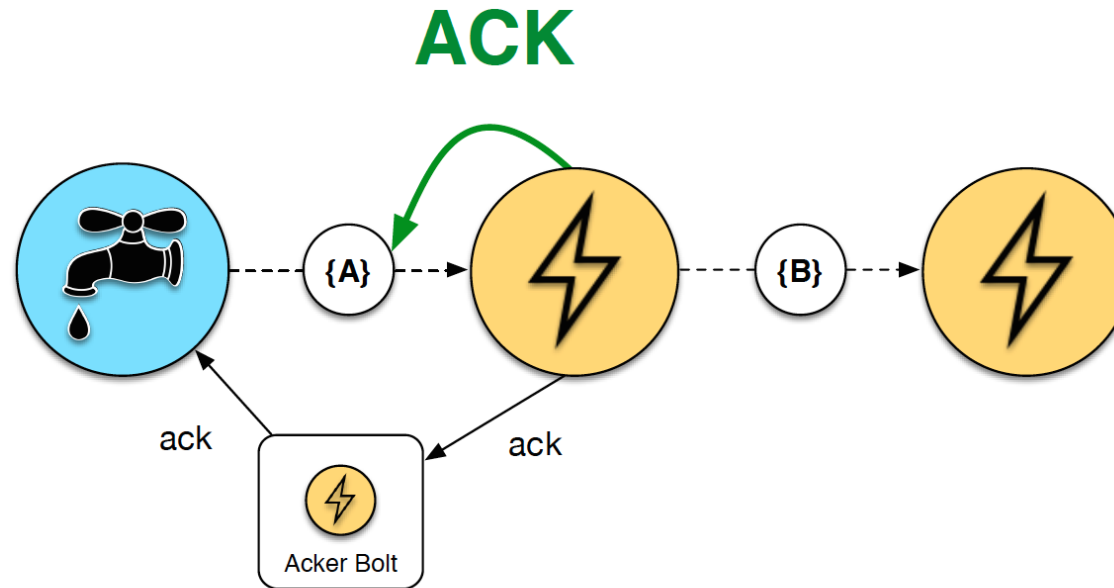
- (Optionally) Emit additional streams

Topologies



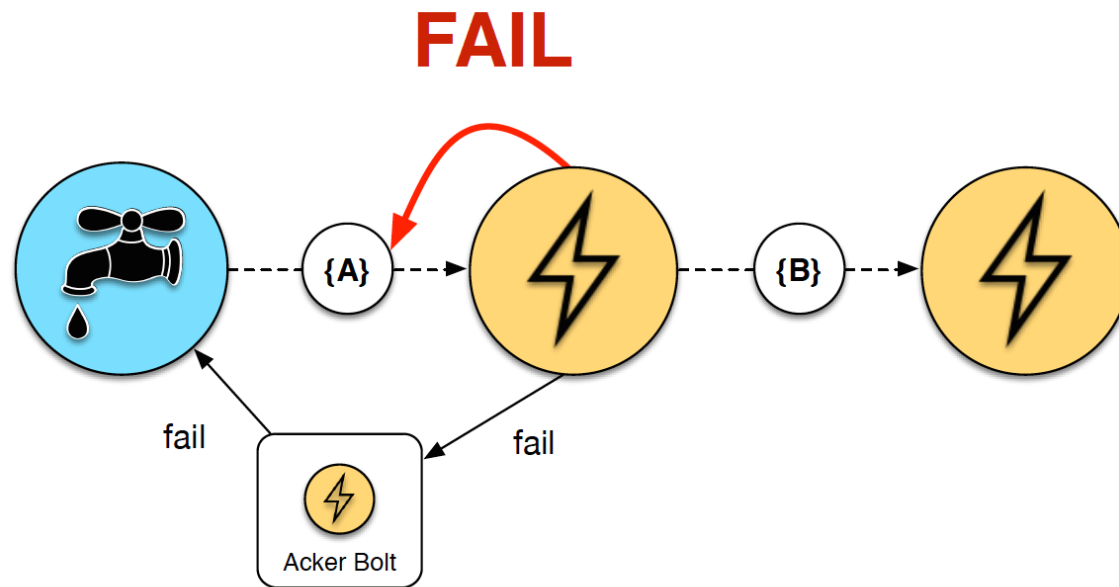
- DAG of Spouts and Bolts
- Data Flow Representation
- *Streaming Computation*

Reliable Processing



Acks are delivered via a system-level bolt

Reliable Processing

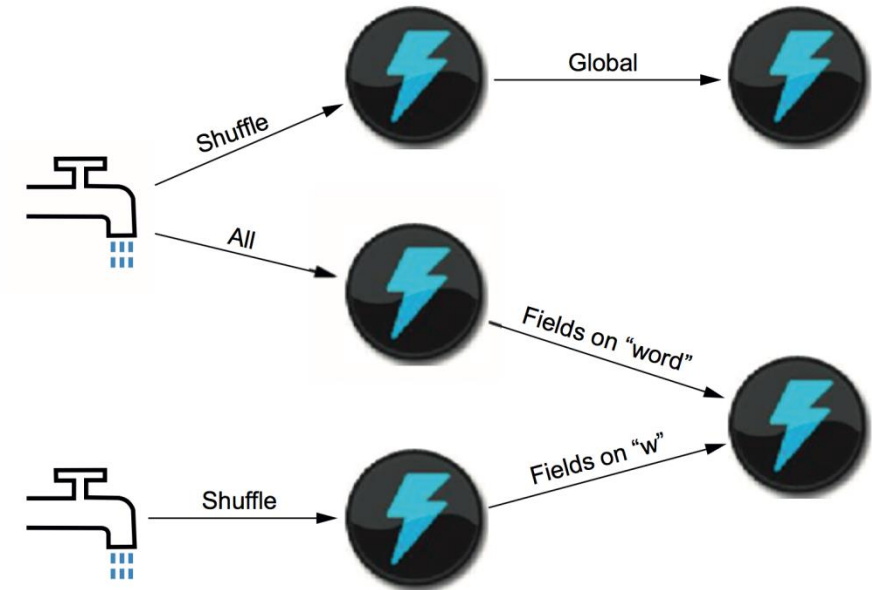


Bolts can also ***Fail*** a tuple to trigger a spout to replay the original.

Stream Grouping

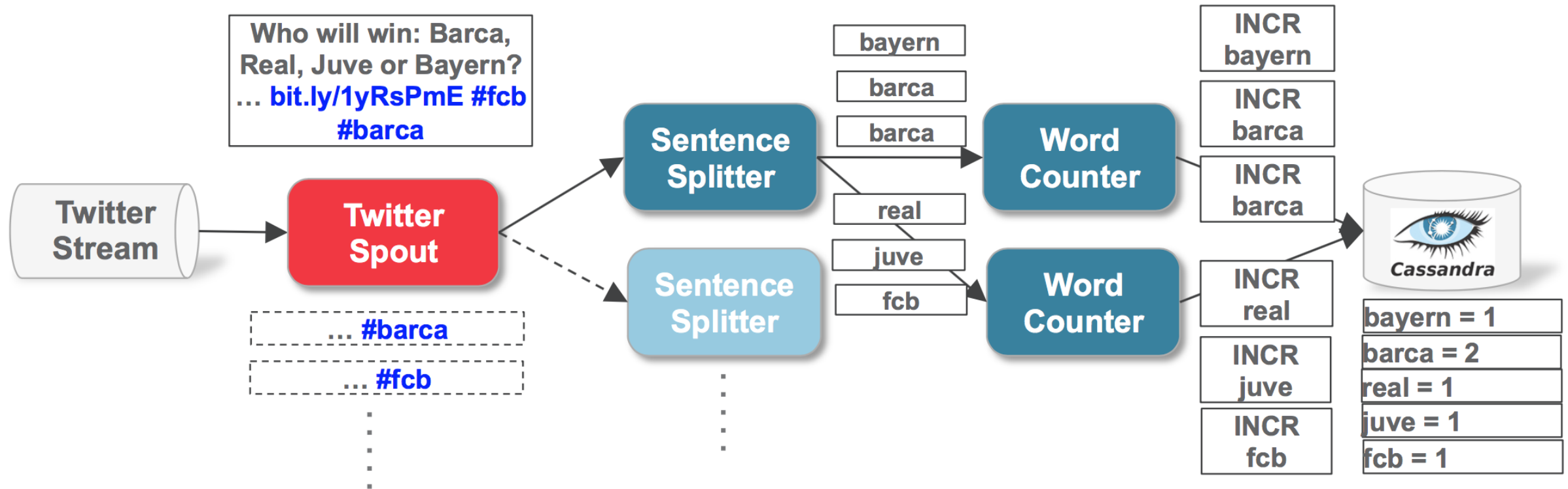


- **Shuffle grouping:** is random grouping
- **Fields grouping:** is grouped by value, such that equal value results in equal task
- **All grouping,** replicates to all tasks
- **Global grouping,** makes all tuples go to one task
- **None grouping,** makes bolt run in the same thread as bolt/spout it subscribes to
- **Direct grouping,** producer (task that emits) controls which consumer will receive
- **Local or shuffle grouping,** shuffle grouping among bolt tasks running in the same worker process, if any.



WHEN A TUPLE IS EMITTED
WHICH TASK DOES IT GO TO?

Words Counter Example



SHUFFLE GROUPING

FIELDS GROUPING

GLOBAL GROUPING